

TRABAJO FIN DE GRADO

Videojuego 2D de Estrategia Marítima Medieval
Ambientado en la época de la Liga Hanseática

Miguel Menéndez Caro

2.º DAM · Colegios Marianistas Santa Ana y San Rafael

Tutor: Alejandro Jiménez Vitoria

Plantilla de Entrega Parcial

Cómo usar esta plantilla

Este documento está pensado para que lo rellenes tú directamente en tu procesador de texto preferido (o en una nueva versión generada). Cada sección indica claramente qué está ya cubierto con información técnica del proyecto y qué necesitas completar tú.

Fondo	Significado
Fondo blanco / texto normal	Información ya rellena — generada a partir del proyecto real.
Fondo amarillo / cursiva	Campo pendiente de rellenar por ti.
Fondo verde	Apartado de IPE/FOL — remite al documento que estás preparando.
Fondo azul claro	Nota o aclaración de contexto para ayudarte.

■ Para profundizar en cualquier aspecto técnico (clases, métodos, decisiones de arquitectura), consulta el archivo *FEATURES.md* en la raíz del repositorio.

1. Documentación técnica y avance del proyecto

1.1 Memoria inicial del proyecto

HanseBeta es un videojuego de estrategia económica y marítima en 2D desarrollado en Unity 6 con C# como lenguaje principal. El jugador asume el papel de un mercader en la época de la Liga Hanseática, gestionando una flota de barcos para comerciar entre seis ciudades portuarias europeas —Venecia, Génova, Barcelona, Ruan, Lübeck y Brujas— con el objetivo de maximizar su patrimonio a través de la compra y venta de mercancías.

La referencia de jugabilidad principal es la saga **Patrician** (especialmente Patrician III/IV): simulación económica profunda con precios reactivos a la oferta y la demanda, cadenas de producción entre bienes y combates navales. El proyecto se desarrolla como Trabajo Fin de Grado del ciclo 2.º DAM.

Estado actual del desarrollo

■ Esta sección resume el estado en el momento de la entrega. Para el desglose día a día, consulta los archivos `dia_N_memoria.md` del repositorio.

Módulo	Estado	Observaciones
Módulo económico (mercado, precios, stock)	Completado	Precios reactivos, fórmula oferta/demanda, compra/venta funcional.
Módulo de ciudades (6 ciudades + edificios)	Completado	Vista 2D con mercado, taberna y astillero por ciudad.
Base de datos SQLite (15 tablas, 5 slots)	Completado	Schema completo con DAOs, SaveManager y LoadManager.
Módulo de tiempo y simulación	Completado	Tick diario, velocidades 0.25x–10x y pausa.
Mapamundi hexagonal con pathfinding	Completado	Tilemap hex, A*, niebla de guerra, flotas PNJ en movimiento.
PNJs (comerciantes y piratas)	Completado	Máquinas de estados, memoria de precios con retraso 7 días.
Combate naval automático	Completado	Resolución por estadísticas: vida, ataque, escolta.
Astillero funcional + taberna	Completado	Compra, mejora y gestión de tripulación.
Audio (música + efectos)	En curso	Integración de clips por escena y efectos básicos.
Pulido visual (sprites, iconos finales)	En curso	Iconos de bienes, sprites diferenciados por tipo de casco.
Build final Windows standalone	Pendiente	Code freeze: 25 de mayo de 2026.
Combate naval manual (tablero grid)	Post-TFG	Arquitectura diseñada; fuera del alcance de esta entrega.
Sistema de abordaje	Post-TFG	BD preparada; implementación tras la entrega.

Descripción del problema que resuelve

El mercado de videojuegos de estrategia económica medieval para PC carece de títulos modernos que combinen simulación mercantil profunda con narrativa histórica de la Liga Hanseática. HanseBeta propone un prototipo jugable que demuestra la viabilidad técnica de un motor económico reactivo, persistencia por SQLite y un bucle jugable completo dentro de las restricciones de un proyecto académico de 2.º DAM.

Alcance de esta entrega parcial

Describe brevemente qué funcionalidades están operativas en la build que entregas y qué queda fuera del alcance de esta fase.

1.2 Anexos con contenido complementario

Los siguientes documentos técnicos están disponibles como anexos del proyecto y pueden adjuntarse o referenciarse desde esta memoria:

Anexo	Descripción	Ubicación en el repo
Anexo 1 — ERD Schema SQLite	Diagrama entidad-relación con las 15+ tablas de la base de datos, campos, tipos y relaciones.	TFG/Database/erd_schema_dia8.md + erd_schema_dia8.drawio
Anexo 2 — FEATURES.md	API pública del proyecto: todas las clases, métodos, decisiones de diseño y TO-DOs.	TFG/FEATURES.md
Anexo 3 — Capturas de pantalla	Capturas de cada escena del juego para ilustrar el estado visual del proyecto.	[Adjuntar capturas del día 31]
Anexo 4 — Diagrama de Gantt	Cronograma real vs planificado, generado a partir del calendario de desarrollo.	Calendario_TFG.md / [Exportar como imagen]
Anexo 5 — Demo en vídeo	Grabación de 3-5 minutos mostrando el bucle jugable completo.	[Enlace o fichero adjunto]

1.3 Cumplimiento de la planificación

1.3.1 Comparación entre lo planificado y los avances reales

■ La planificación inicial se estableció en la sesión de release (sesion_planificacion_release.md). El calendario completo con las desviaciones reales está en Calendario_TFG.md.

Semana	Previsto	Real	Estado
1 (24-27 abr)	Módulo tiempo + tick ciudades + 4 ciudades nuevas	Completado según plan	✓ OK
2 (28 abr – 4 may)	SQLite completo: DAOs, Save/Load, pantalla slots	Completado según plan	✓ OK
3 (5-11 may)	PNJs + astillero + taberna + cámara mapamundi	Completado según plan	✓ OK

4 (12-18 may)	Combate naval completo	Combate automático completo; manual pospuesto a post-TFG	■ Ajustado
5 (19-25 may)	Tilemaps + pulido + audio + documentación + build	En curso	■ En progreso
Testing (26-31 may)	Solo bugfixes + memoria final	Pendiente	■ Pendiente

1.3.2 Justificación de ajustes al cronograma

La principal desviación respecto al plan inicial fue la decisión de implementar únicamente el **combate naval automático** en lugar del manual. El módulo de combate manual (tablero grid, turnos por velocidad, ataque por secciones) requería un volumen de trabajo incompatible con el time-box disponible. La arquitectura de datos ya contempla ambas modalidades —la BD incluye secciones de barco, tipos de armamento y tripulación— por lo que la extensión a combate manual queda documentada y preparada para el desarrollo post-TFG.

Esta decisión está documentada en el apartado de justificación del Calendario_TFG.md y en la sección correspondiente de FEATURES.md bajo el epígrafe "Lo que NO entra en el TFG".

1.4 Producto Mínimo Viable (PMV) y mejoras

Definición del PMV

El PMV de HanseBeta es una partida completa y persistente que incluye: creación de nueva partida, navegación por el mapamundi hexagonal, entrada en cualquiera de las seis ciudades, compra y venta de mercancías en el mercado con precios dinámicos, encuentro y resolución de un combate naval automático, guardado y carga de la partida.

Funcionalidad mínima	Incluida en PMV
Nueva partida + selección de ciudad de inicio	Sí
Mercado con precios reactivos (19 bienes)	Sí (5 bienes en beta, 19 en release)
Navegación por mapamundi hexagonal con A*	Sí
Encuentro con pirata y combate automático	Sí
Guardar y cargar partida (5 slots)	Sí
PNJs comerciantes y piratas con IA básica	Sí
Astillero y taberna funcionales	Sí
Combate naval manual (tablero grid)	No — post-TFG
Sistema de abordaje	No — post-TFG

Mejoras planificadas post-TFG

Describe brevemente las mejoras que te gustaría añadir tras la entrega (combate manual, abordaje, edificios construibles, etc.) y su impacto esperado en la jugabilidad.

1.5 Presupuesto y análisis de costes

■ **Pendiente IPE/FOL:** Rellena este apartado con los datos de tu documento IPE/FOL: horas estimadas, coste por hora junior/senior, herramientas de pago si las hay, hardware de desarrollo.

■ **Referencia rápida:** Unity Personal es gratuito para proyectos sin ingresos superiores a 100.000 \$/año. VS Code, Git y SQLite son gratuitos. El hardware necesario es un ordenador de desarrollo estándar.

Concepto	Coste unitario	Cantidad	Total
Desarrollador (tú)	[€/hora]	[X horas]	[Total €]
Unity Personal	0 €	—	0 €
Visual Studio Code	0 €	—	0 €
SQLite	0 €	—	0 €
Assets gráficos (Kenney.nl)	0 € (licencia CC0)	—	0 €
Hardware de desarrollo	[Amortización]	—	[€]
Otros (especificar)	[€]	[—]	[€]

1.6 Plan de viabilidad básico

■ **Pendiente IPE/FOL:** Completa con tu análisis de mercado objetivo, riesgos y factores económicos del documento IPE/FOL.

Mercado objetivo

Describe el perfil del jugador objetivo: edad, plataforma, gustos (estrategia, historia medieval, trading games), disposición a pagar, etc.

Riesgos y estrategias de mitigación

Riesgo identificado	Probabilidad	Estrategia de mitigación
[Describe riesgo 1]	[Alta/Media/Baja]	[Cómo mitigarlo]
[Describe riesgo 2]	[Alta/Media/Baja]	[Cómo mitigarlo]
[Describe riesgo 3]	[Alta/Media/Baja]	[Cómo mitigarlo]

1.7 Licencia del software creado

Indica la licencia elegida para el proyecto (MIT, GPL, Creative Commons, etc.) y justifica brevemente tu elección. Recuerda añadir un archivo LICENSE.txt en la raíz del repositorio.

■ *Recomendación del proyecto: licencia MIT. Permite uso, modificación y distribución libre manteniendo el crédito al autor original, lo que facilita la evaluación académica y futura colaboración.*

2. Aspectos técnicos del desarrollo

2.1 Definición de objetivos

Objetivo principal

Redacta en tus propias palabras el objetivo principal del proyecto: qué problema resuelve, para quién y qué resultado final se espera obtener.

Objetivos específicos

A continuación se listan los objetivos técnicos derivados del diseño del sistema:

Objetivo específico	Criterio de éxito
Implementar un motor económico con precios reactivos a oferta/demanda.	El precio de cada bien varía dinámicamente al comprar/vender. Verificable en cualquier partida.
Desarrollar un sistema de persistencia por SQLite con 5 slots de guardado.	La partida se guarda y carga sin pérdida de datos en todas las tablas del schema.
Construir un mapamundi navegable con pathfinding A* sobre tilemap hexagonal.	El jugador puede trazar rutas entre ciudades; los PNJs se mueven de forma autónoma.
Integrar IA básica para PNJs comerciantes y piratas.	Los comerciantes operan con precios con 7 días de retraso; los piratas patrullan y atacan.
Implementar combate naval con resolución automática.	Dos flotas se enfrentan con resultado determinado por sus estadísticas, sin intervención del jugador.
[Añade objetivos propios]	[Criterio de éxito]

2.2 Análisis de competencia

■ Para profundizar en este análisis consulta la sección 'Análisis de competencia' de la planificación de release (sesion_planificacion_release.md).

Título	Plataforma	Año	Puntos fuertes	Carencias respecto a HanseBeta
Patrician IV	PC	2010	Simulación económica profunda, rutas comerciales, gremios.	Motor antiguo, sin desarrollo activo, UX obsoleta.
Port Royale 4	PC / Consola	2020	Gráficos modernos, combate naval.	Simulación económica superficial, sin construcción de ciudades.
Anno 1404	PC	2009	Construcción y gestión de ciudades muy detallada.	Sin comercio entre ciudades históricas europeas medievales; enfoque colonial.
[Añade competidor]	[—]	[—]	[Puntos fuertes]	[Carencias]

Propuesta diferencial de HanseBeta

Explica en un párrafo qué hace único a tu proyecto respecto a los competidores identificados. Puedes mencionar el ambientación específica en la Hansa, el nivel técnico del motor económico, la profundidad del sistema de persistencia, etc.

2.3 Recursos software, hardware y justificación de la solución

2.3.1 Software de desarrollo

Herramienta	Versión	Uso	Coste	Justificación
Unity	6 (LTS)	Motor de juego — proyecto 2D URP	Gratuito (Personal)	Estándar de la industria para juegos 2D/3D; amplia comunidad y documentación.
C#	12 (.NET 8)	Lenguaje de programación principal	Gratuito	Lenguaje nativo de Unity; fuertemente tipado, ideal para arquitectura de juego.
SQLite	3.x	Base de datos embebida por partida	Gratuito / Open Source	Sin servidor, un fichero .db por partida, perfecto para persistencia local en juegos.
Visual Studio Code	Última estable	Editor de código con extensión C#	Gratuito	Ligero, rápido y con buen soporte de IntelliSense para C# via OmniSharp.
TextMesh Pro	Incluido en Unity	Renderizado de texto en UI	Gratuito	Renderizado vectorial de alta calidad; alternativa superior al Text nativo de Unity.
Git + GitHub	—	Control de versiones + alojamiento	Gratuito	Una rama por día de trabajo; historial completo del desarrollo.

2.3.2 Hardware de desarrollo

Describe el equipo que has usado para desarrollar el proyecto: procesador, RAM, tarjeta gráfica (si aplica), sistema operativo. Indica si es propio o del centro.

2.3.3 Justificación de la solución tecnológica

La elección de **Unity 6** como motor responde a tres factores: es la plataforma de desarrollo de juegos más usada en el ámbito académico y profesional para el perfil 2D/3D independiente; su versión Personal es completamente gratuita para proyectos sin ingresos; y el equipo ya contaba con experiencia previa en C#.

La elección de **SQLite** en lugar de un servidor de base de datos externo (PostgreSQL, MySQL) se justifica por la naturaleza local del juego: cada partida es un fichero .db independiente, sin necesidad de red, con un esquema relacional completo de 15+ tablas que garantiza la integridad referencial y la reconstrucción fiel del estado del mundo al cargar.

2.4 Planificación y metodología de desarrollo

2.4.1 Metodología seleccionada

El proyecto se ha desarrollado siguiendo un modelo **iterativo por sprints semanales**, similar a Scrum pero adaptado al trabajo individual. Cada semana constituye un sprint con objetivos claros, y cada día de trabajo

genera una rama Git independiente (día-N) que se fusiona a main mediante Pull Request al final de la jornada.

Esta metodología garantiza trazabilidad completa del avance, facilita la detección temprana de bloqueos y permite replanificar con información real (como ocurrió con el combate naval manual en la semana 4).

2.4.2 Diagrama de Gantt

Inserta aquí el diagrama de Gantt exportado desde Calendario_TFG.md. Puedes generarlo en Google Sheets, Canva o GanttProject a partir de las fechas del calendario y adjuntarlo como imagen.

■ El calendario completo con fechas reales está en Calendario_TFG.md del repositorio. Cada entrada incluye modo (code/diseño/mixto), bloque temático y entregables concretos del día.

2.5 / 2.6 Diseño visual y experiencia de usuario (UX)

2.5.1 Introducción al diseño visual

HanseBeta es un juego 2D con vistas de ciudad planas y mapamundi hexagonal. La paleta visual se basa en tonos tierra, azul mar y dorado para evocar la estética medieval marítima. La interfaz prioriza la legibilidad de los datos económicos (precios, stock, dinero del jugador) sobre el aspecto decorativo, coherente con el género de estrategia de gestión.

2.5.2 Capturas de pantalla

Inserta aquí capturas de las escenas principales: Menú Principal, Vista de Ciudad, Mercado, Mapamundi con flotas, Pantalla de Combate y Pantalla de Resultados. Referencia: capturas del Día 31 del repositorio.

2.5.3 Flujo de navegación entre pantallas

El flujo de pantallas del juego es lineal con bifurcaciones contextuales:

Pantalla	Acceso desde	Acciones principales
Menú Principal	Inicio de la aplicación	Nueva partida, cargar partida (5 slots), salir.
Selección de ciudad de inicio	Nueva partida	Elegir entre las 6 ciudades disponibles.
Vista de Ciudad	Menú Principal / Mapamundi al llegar a puerto	Entrar al Mercado, Astillero o Taberna; zarpar al mapamundi.
Mercado	Vista de Ciudad	Consultar precios y stock; comprar y vender bienes.
Astillero	Vista de Ciudad	Comprar/mejorar barcos; gestionar flota.
Taberna	Vista de Ciudad	Contratar tripulación; recibir rumores de mercado.
Mapamundi	Vista de Ciudad al zarpar	Navegar entre ciudades; encontrar flotas PNJ; iniciar combates.
Pantalla de Combate	Mapamundi al encontrar pirata	Observar resolución automática del combate.

Pantalla de Resultados	Tras el combate	Ver botón, bajas y decisión de continuar.
------------------------	-----------------	---

2.7 Base de datos — Esquema relacional

2.7.1 Introducción

La base de datos de HanseBeta es un fichero SQLite independiente por partida. Este enfoque garantiza que cada partida es un artefacto autocontenido, sin dependencias de red ni servidor externo. El schema cuenta con **15 tablas relacionales** que modelan el estado completo del mundo del juego: jugador, flotas, barcos, ciudades, mercados, edificios, PNJs y el historial de precios de los comerciantes.

2.7.2 Esquema relacional

■ El ERD completo en formato Mermaid (renderizable en GitHub) está en `TFG/Database/erd_schema_dia8.md`. La versión visual editable está en `erd_schema_dia8.drawio`. Adjunta el diagrama como imagen en este apartado.

Inserta aquí la imagen del ERD exportada desde [diagrams.net](#) o [GitHub](#).

2.7.3 Descripción de las tablas principales

Tabla	Descripción	Relaciones clave
estadoJuego	Registro único con el estado global de la partida: dinero del jugador, fecha del juego (día/mes/año), velocidad de simulación.	Independiente — sin FK salientes.
Ciudad	Catálogo de las 6 ciudades con nombre, coordenadas en el tilemap y especialización productiva.	1:N con EstadoMercadoCiudad, EdificiosCiudad, Flota (ciudad_destino).
Bien	Catálogo de los 19 bienes del juego con categoría (primario/intermedio/avanzado) y precio base.	1:N con EstadoMercadoCiudad; N:M reflexivo via RecetaProduccion.
EstadoMercadoCiudad	Stock actual y precio calculado de cada bien en cada ciudad. Se actualiza en cada tick diario.	FK a Ciudad y Bien. Clave compuesta (id_ciudad, id_bien).
RecetaProduccion	Tabla N:M reflexiva sobre Bien que modela las cadenas de producción: qué bienes se necesitan para producir otro bien.	FK doble a Bien: id_bien_resultado e id_bien_ingredient.
TipoCasco	Catálogo de los 4 tipos de casco de barco con estadísticas base: vida, ataque, capacidad_bodega, velocidad_base.	1:N con Barco.
Capitan	Personajes con nombre y habilidades que dirigen flotas. Un barco hundido pierde la flota.	1:1 con Flota (la flota requiere capitán).
Flota	Agrupación de hasta 5 barcos. Tiene posición actual en el tilemap, estado de la máquina de estados (EnPuerto/Navegando/Combatiendo) y ciudad de destino.	1:N con Barco; FK a Ciudad (ubicacion_actual, id_ciudad_destino); FK a Capitan.

Barco	Cada barco de una flota con su tipo, nombre, vida actual y capacidad de tripulación.	FK a Flota y TipoCasco. 1:N con CargaBarco y EstadoSeccionesBarco.
CargaBarco	Mercancía que transporta cada barco (N:M entre Barco y Bien con cantidad).	FK a Barco y Bien. Clave compuesta (id_barco, id_bien).
MemoriaComercialPNJ	Precios que conoce cada flota PNJ de cada bien, con el día de juego en que los aprendió. Simula el retraso de 7 días.	FK a Flota y Bien. Clave compuesta.
AlmacenJugador	Bienes que posee el jugador en su almacén global (beta) o por ciudad (release).	FK a Bien.

2.7.4 Ficheros XML

El proyecto no utiliza ficheros XML para el almacenamiento de datos de partida. Los **ScriptableObject**s de **Unity** (archivos .asset serializados por el motor) actúan como catálogo estático de bienes y ciudades, separando la configuración del juego de los datos de la partida almacenados en SQLite. Esta separación permite modificar el catálogo sin alterar las partidas guardadas.

Notas finales y firma

Esta plantilla cubre todos los apartados exigidos en las instrucciones de entrega parcial del TFG. Los campos en amarillo deben ser completados por el alumno. Los apartados en verde corresponden al módulo IPE/FOL y deben integrarse desde el documento específico que se está preparando.

■ Recuerda revisar ortografía, completar los anexos referenciados y exportar el documento final con el nombre TFG_Parcial1_MenéndezCaroMiguel.pdf (o .docx según lo que indique el tutor).

Alumno	Tutor	Centro	Fecha de entrega
Miguel Menéndez Caro	Alejandro Jiménez Vitoria	Colegios Marianistas Santa Ana y San Rafael	[Fecha indicada por el tutor]